智能信息处理

刁云峰
计算机学院智能科学系
diaoyunfeng@hfut.edu.cn

第一节 什么是深度学习框架

什么是深度学习框架

深度学习采用“端到端”的学习模式，在很大程度上减轻负担。但随着神经网络的发展，模型的复杂度也在不断提升。

框架存在的意义就是屏蔽底层的细节，使研究者可以专注于模型结构。

Anaconda 安装

Anaconda是Python的一个开源发行版本，主要面向科学计算。我们可以简单理解为，Anaconda是一个预装了很多我们用的到或用不到的第三方库的Python。而且相比于大家熟悉的pip install命令，Anaconda中增加了conda install命令。当你熟悉了Anaconda以后会发现，conda install会比pip install更方便一些

总的来说，我们应该完成以下几步：

•根据操作系统下载并安装Anaconda（或者mini版本Miniconda）并学会常用的几个conda命令，例如如何管理python环境、如何安装卸载包等；

•Anaconda安装成功之后，我们需要修改其包管理镜像为国内源，这样以后安装包时就会快一些。

最省心的Python版本和第三方库管理——初探Anaconda

初学 Python 者自学 Anaconda 的正确姿势-猴子的回答。

https://www.anaconda.com/

ANACONDA. Products Solutions Resources Partners Company Sign Up Sign In

Download Now

For installation assistance, refer to Troubleshooting.

Download Anaconda Distribution or Miniconda by choosing the proper installer for your machine. Learn the difference from our Documentation.

Anaconda Installers

Download

Windows

Python 3.12

- 64-Bit Graphical Installer (912.3M)

Mac

Python 3.12

- 64-Bit (Apple silicon) Graphical Installer (704.7M)
- 64-Bit (Apple silicon) Command Line Installer (707.3M)
- 64-Bit (Intel chip) Graphical Installer (734.7M)
- 64-Bit (Intel chip) Command Line Installer (731.2M)

Linux

Python 3.12

- 64-Bit (x86) Installer (1007.9M)
- 64-Bit (AWS Graviton2 / ARM64) Installer (800.6M)
- 64-bit (Linux on IBM Z & LinuxONE) Installer (425.8M)

如何用Anaconda管理你的Python?

第1步房子设计

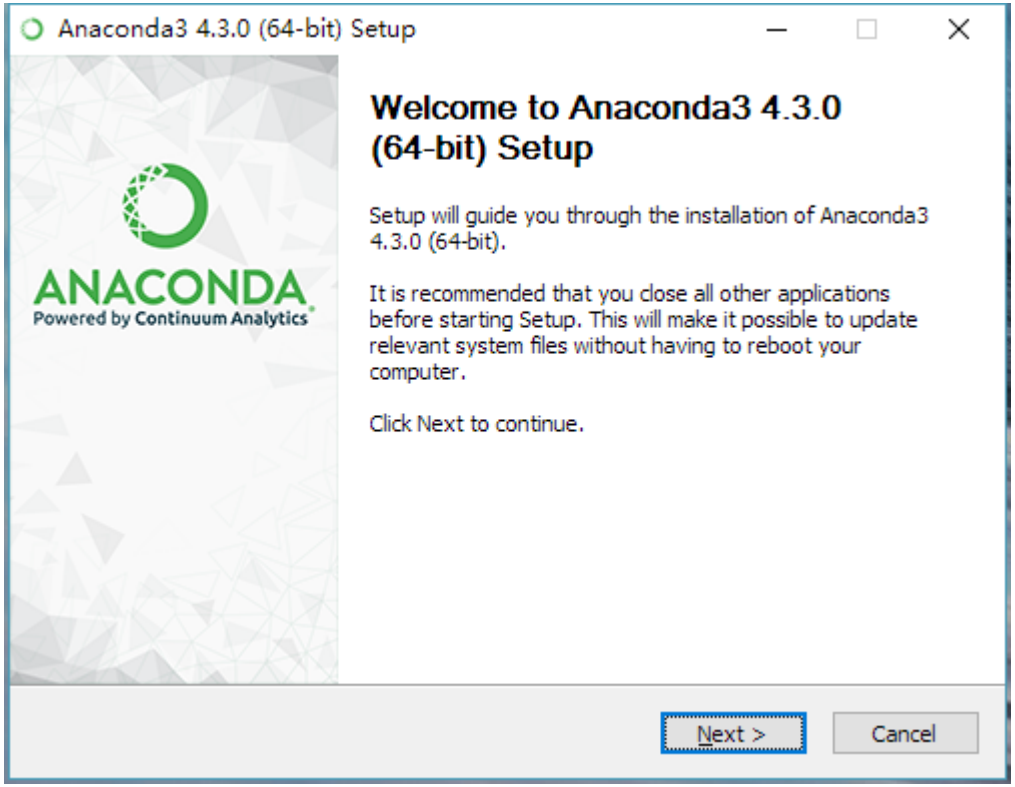
第2步地基
(运行环境Anaconda)

第3步准备材料
(数据)

第4步组合材料
(Python数据分析)

猴子

https://www.anaconda.com/


Anaconda3 4.3.0 (64-bit) Setup
ANACONDA
Powered by Continuum Analytics
Welcome to Anaconda3 4.3.0 (64-bit) Setup
Setup will guide you through the installation of Anaconda3 4.3.0 (64-bit).
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.
Click Next to continue.
Next >
Cancel

Numpy的使用

- Numpy（Numerical Python）是Python的一个开源数值计算扩展库。
- 提供高性能的多维数组对象及相关工具。
- 广泛用于科学计算、数据分析、机器学习等领域。

第二节 PyTorch的特点和概述

本章目录

01 NumPy概述

02 NumPy数组(ndarry)对象

03 ufunc函数

04 NumPy的函数库

1.NumPy概述

01 NumPy概述

02 NumPy数组(ndarry)对象

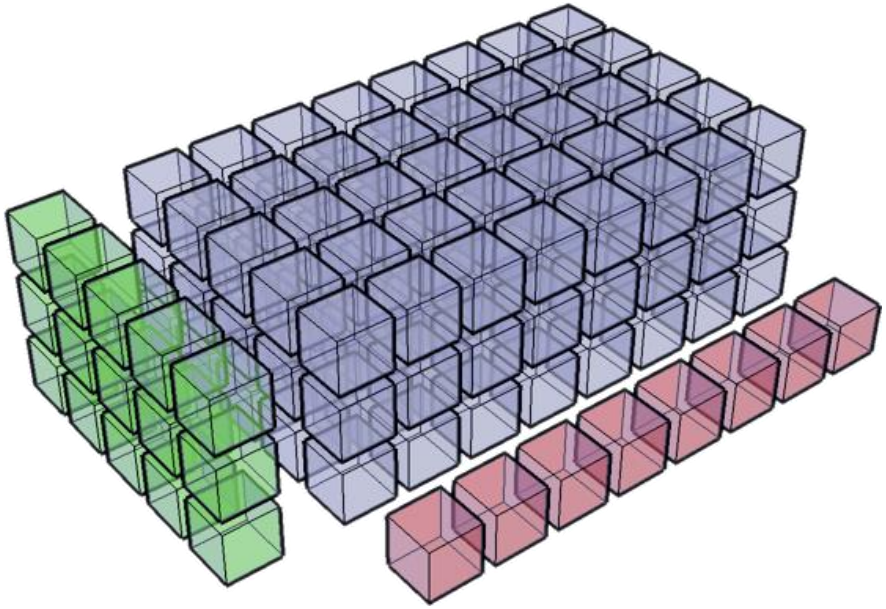
03 ufunc函数

04 NumPy的函数库

NumPy是什么？

NumPy(Numeric Python)是Python的一种开源的数值计算扩展库。
它包含很多功能：

· 创建n维数组（矩阵）
· 对数组进行函数运算
· 数值积分
· 线性代数运算
· 傅里叶变换
· 随机数产生
……



NumPy是什么？

NumPy提供了许多高级的数值编程工具，如：矩阵数据类型、矢量处理，以及精密的运算库。专为进行严格的数字处理而产生。多为很多大型金融公司使用，以及核心的科学计算组织如：Lawrence Livermore，NASA 用其处理一些本来使用 C++，Fortran 或 Matlab 等所做的任务。

NumPy是什么？

标准的**Python**中用**list**（列表）保存值，可以当做数组使用，但因为列表中的元素可以是任何对象，所以浪费了CPU运算时间和内存。

NumPy诞生为了弥补这些缺陷。它提供了两种基本的对象：

ndarray：全称（n-dimensional array object）是储存单一数据类型的多维数组。

ufunc：全称（universal function object）它是一种能够对数组进行处理的函数。

NumPy的官方文档：

https://docs.scipy.org/doc/numpy/reference/

NumPy

NumPy的安装

Anaconda里面已经安装过NumPy。

原生的Python安装：

· 在cmd中输入

```
pip install numpy
```

安装之后，我们用**导入**这个库

```
import numpy as np
```

2.NumPy数组(ndarry)对象

01 NumPy概述
02 NumPy数组(ndarry)对象
03 ufunc函数
04 NumPy的函数库

1.1 认识 NumPy 数组对象

NumPy 最重要的一个特点是其 N 维数组对象 **ndarray**，它是一系列同类型数据的集合，以 0 下标为开始进行集合中元素的索引。

ndarray 对象是用于存放同类型元素的多维数组。

```
>import numpy as np # 导入NumPy工具包
>data = np.arange(12).reshape(3, 4) # 创建一个3行4列的数组
>data
```

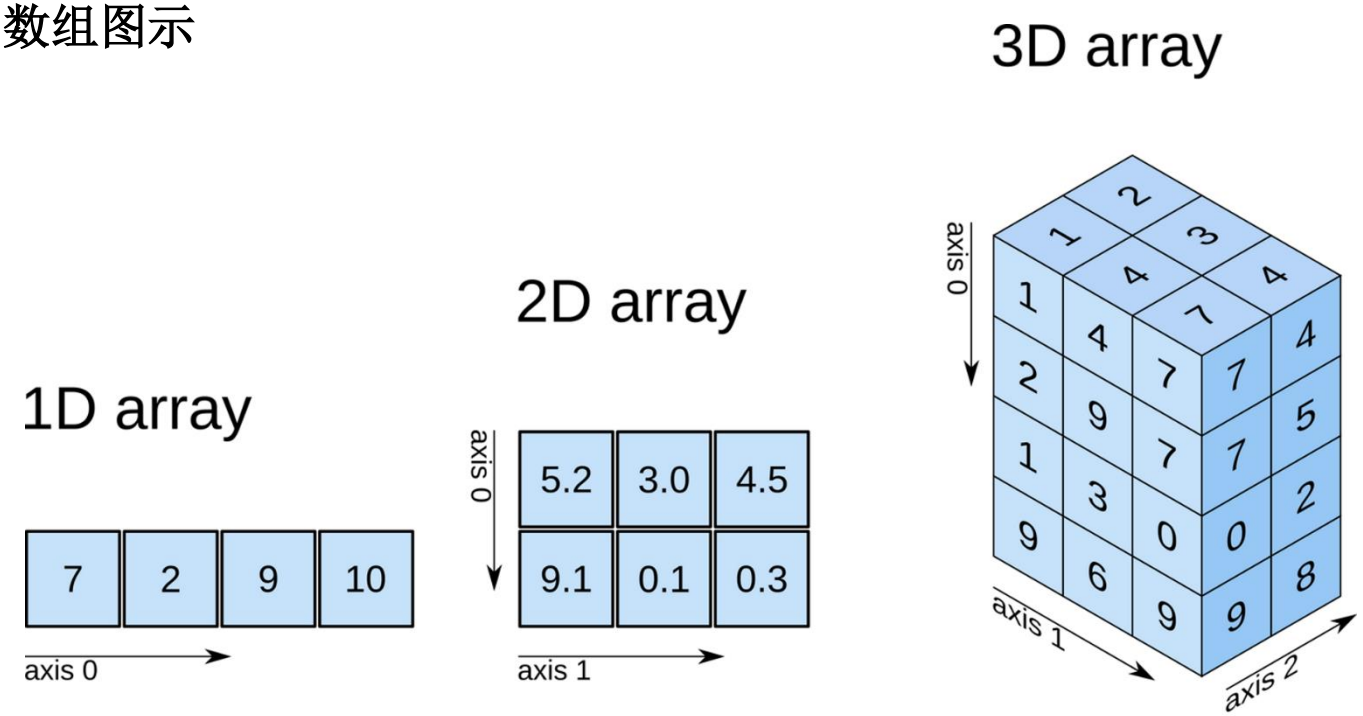
```
array([[ 0, 1, 2, 3],
       [ 4, 5, 6, 7],
       [ 8, 9, 10, 11]])
```

ndarray对维数没有限制。

[ ]从内到外分别为第0轴，第1轴，第2轴，第3轴。

1.1 认识 NumPy 数组对象

NumPy 数组图示



shape(4,) shape(3,2) shape(4,3,2)

1.1 认识 NumPy 数组对象

```
>type(data)
```

numpy.ndarray

```
>data.ndim # 数组维度的个数，输出结果2，表示二维数组
```

2

```
>data.shape # 数组的维度，输出结果(3,4),表示3行4列
```

(3, 4)

```
data.size # 数组元素的个数，输出结果12，表示总共有12个元素
```

12

```
data.dtype # 数组元素的类型，输出结果dtype('int64'),表示元素类型都是int64
```

dtype('int32')

1.2 创建 NumPy 数组

```
>import numpy as np
>data1 = np.array([1, 2, 3]) # 创建一个一维数组
>data1
```

```
array([1, 2, 3])
```

```
data2 = np.array([[1, 2, 3], [4, 5, 6]]) # 创建一个二维数组
data2
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.zeros((3, 4))#创建一个全0数组
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

1.2 创建 NumPy 数组

```
>np.ones((3, 4))#创建全一数组
```

```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

```
>np.empty((5, 2))# 创建全空数组，其实每个值都是接近于零的数
```

```
array([[ 6.95312756e-310,  2.12199579e-314],
       [ 2.12199579e-314,  4.94065646e-324],
       [ 0.00000000e+000, -7.06252554e-311],
       [ 0.00000000e+000, -8.12021073e-313],
       [ 1.29923372e-311,  2.07507571e-322]])
```

1.2 创建 NumPy 数组

```
>np.arange(1, 20, 5)
```

```
array([ 1,  6, 11, 16])
```

```
>np.array([1, 2, 3, 4], float)
```

```
array([1., 2., 3., 4.])
```

```
>np.ones((2, 3), dtype='float64')
```

```
array([[1., 1., 1.],
       [1., 1., 1.]])
```

ndarray的创建

NumPy提供了**专门用于生成ndarray的函数**，提高创建ndarray的速度。

```
> a = np.arange(0, 1, 0.1)
array([ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9])

> b = np.linspace(0, 1, 10)
array([ 0.     ,  0.11111111,  0.22222222,  0.33333333,
0.44444444,0.55555556,  0.66666667,  0.77777778,  0.88888889,  1.     ])

> c = np.linspace(0, 1, 10, endpoint=False)
array([ 0. ,  0.1,  0.2,  0.3,  0.4,  0.5,  0.6,  0.7,  0.8,  0.9])

> d = np.logspace(0, 2, 5)
array([   1. ,    3.16227766,   10.,   31.6227766 ,  100.])
```

ndarray的创建

np.empty((2,3), np.int)	创建2*3的整形型空矩阵，只分配内存
np.zeros(4, np.int)	创建长度为4，值为全部为0的矩阵
np.full(4, np.pi)	创建长度为4，值为全部为pi的矩阵

还可以**自定义函数产生ndarray**。

```
> def func(i):
      return i % 4 + 1
> np.fromfunction(func, (10,))

array([ 1.,  2.,  3.,  4.,  1.,  2.,  3.,  4.,  1.,  2.])
```

fromfunction第一个参数接收计算函数，第二个参数接收数组的形状。

ndarray的属性

ndarray的元素具有相同的元素类型。常用的有int（整型），float（浮点型），complex（复数型）。

```
> a = np.array([1, 2, 3, 4], dtype=float)
array([ 1.,  2.,  3.,  4.])

> a.dtype
dtype('float64')
```

ndarray的shape属性用来获得它的形状，也可以自己指定。

```
> c = np.array([[1, 2, 3, 4], [4, 5, 6, 7], [7, 8, 9, 10]])
> c.shape
(3, 4)

> a = np.array([1, 2, 3, 4])
> d = a.reshape((2,2))
array([[1, 2],
       [3, 4]])
```

ndarray的切片

ndarray的切片和list是一样的。

```
> a = np.arange(10)
> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

a[5]    a[3:5]      a[:5]                a[:-1]
----    ------    ---------------    ---------------------------
5        [3, 4]    [0, 1, 2, 3, 4]      [0, 1, 2, 3, 4, 5, 6, 7, 8]

 a[1:-1:2]              a[::-1]                        a[5:1:-2]
------------    ------------------------------    ---------
[1, 3, 5, 7]      [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]        [5, 3]
```

可以通过切片的对ndarray中的元素进行**更改**。

```
> a[2:4] = 100, 101
> a
array([  0,   1, 100, 101,   4,   5,   6,   7,   8,   9])
```

ndarray的切片

ndarray通过切片产生一个新的数组b，**b和a共享同一块数据存储空间。**

```
> b = a[3:7]
> b[2] = -10
 b                                  a
--------------------  ------------------------------------------------
[101,   4, -10,   6]    [  0,   1, 100, 101,   4, -10,   6,   7,   8,   9]
```

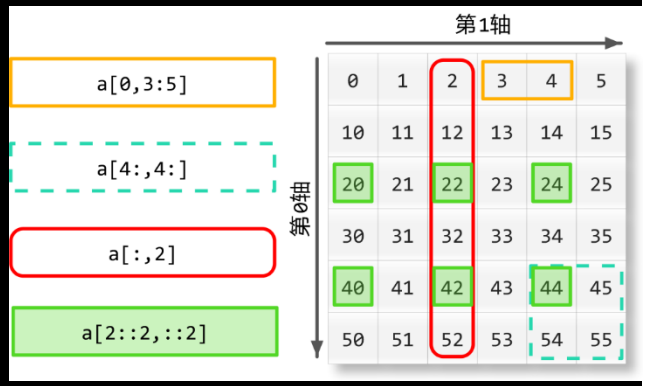
如果想改变这种情况，我们可以**用列表对数组元素切片。**

```
> b = a[[3, 3, -3, 8]]
> b
array([3, 3, 7, 8])

> b[2] = 100
b                                  a
--------------------  ------------------------------------------------
[3,   3, 100,   8]      [  0,   1,  2,  3,   4,  5,   6,   7,   8,   9]
```

多维数组

NumPy的多维数组和一维数组类似。多维数组有多个轴。
我们前面已经提到从内到外分别是第0轴，第1轴...

```
> a = np.arange(0, 60, 10).reshape(-1, 1) + np.arange(0, 6)
array([[ 0,  1,  2,  3,  4,  5],
       [10, 11, 12, 13, 14, 15],
       [20, 21, 22, 23, 24, 25],
       [30, 31, 32, 33, 34, 35],
       [40, 41, 42, 43, 44, 45],
       [50, 51, 52, 53, 54, 55]])

a[0, 3:5]      a[4:, 4:]      a[2::2, ::2]
---------      ----------     --------------
[3, 4]         [[44, 45],     [[20, 22, 24],
                [54, 55]]      [40, 42, 44]]
```

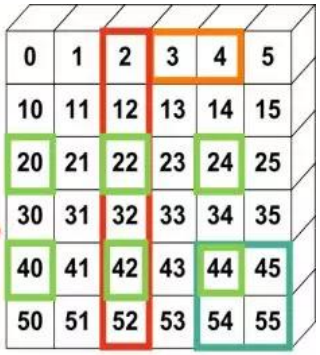


#上面方法对于数组的切片都是共享原数组的储存空间的。

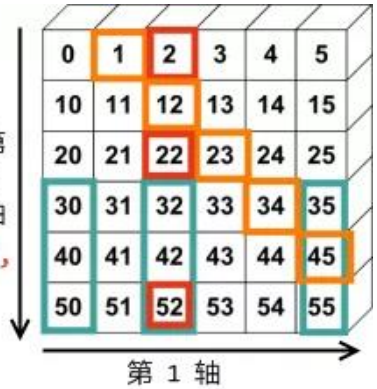
多维数组

如果我们想**创立原数组的副本**，我们可以用**整数元组，列表，整数数组，布尔数组**进行切片。

```
>>> a[0,3:5]
array([3,4])
>>> a[4:,4:]
array([[44,45],[54,55]])
>>> a[:,2]
array([2,12,22,32,42,52])
>>> a[2::2,::2]
array([[20,22,24],
       [40,42,44]])
```




```
>>> a[(0,1,2,3,4),(1,2,3,4,5)]
array([1,12,23,34,45])
>>> a[3:,[0,2,5]]
array([[30,32,35],
       [40,42,45],
       [50,52,55]])
>>> mask=np.array([1,0,1,0,0,1],
                  dtype=np.bool)
>>> a[mask,2]
array([2,22,52])
```



结构数组

C语言中可以通过struct关键字定义结构类型。NumPy中也有类似的**结构数组**。

```
> persontype = np.dtype({
    'names':['name', 'age', 'weight'],
    'formats':['S30','i', 'f']})

> a = np.array([("Zhang", 32, 75.5), ("Wang", 24, 65.2)],
      dtype=persontype)
```

	name	age	weight
0	zhang	32	75.5
1	wang	24	65.2

我们就创建了一个结构数组，并且可以通过索引得到每一行。

```
> print a[0]
('Zhang', 32, 75.5)
```

4. NumPy的函数库

01 NumPy概述

02 NumPy数组(ndarry)对象

03 ufunc函数

04 NumPy的函数库

随机数

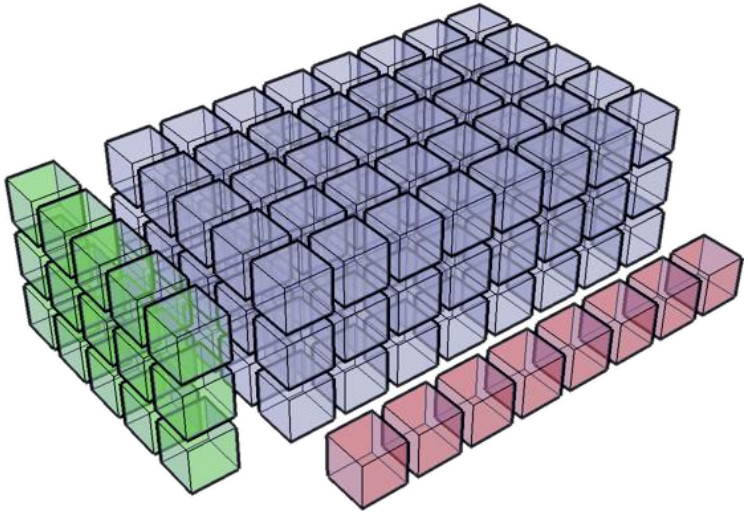
除了前面介绍的ufunc()函数之外，NumPy还提供了大量对于数组运算的函数。它们能够简化逻辑，提高运算速度。

我们首先看随机数。NumPy产生随机数的模块在**random里面，其中有大量的分布。**

```
> from numpy import random as nr
> np.set_printoptions(precision=2) #显示小数点后两位数

> r1 = nr.rand(4, 3)
[[ 0.87,  0.42,  0.34],
[ 0.25,  0.87,  0.42],
[ 0.49,  0.18,  0.44],
[ 0.53,  0.23,  0.81]]

> r2 = nr.poisson(2.0, (4, 3))
[[3, 1, 5],
[2, 2, 3],
[2, 4, 4],
[2, 2, 3]]
```

随机数

rand	0到1之间的随机数	normal	正态分布的随机数
randint	制定范围内的随机整数	uniform	均匀分布
randn	标准正态的随机数	poisson	泊松分布
choice	随机抽取样本	shuffle	随机打乱顺序



求和，平均值，方差

NumPy在均值等方面常用的函数如下：

函数名	功能
sum	求和
average	加权平均数
var	方差
mean	期望
std	标准差
product	连乘积

```
> np.random.seed(42)
> a = np.random.randint(0,10,size=(4,5))

> np.sum(a)
96
```

求和，平均值，方差

```
a                          np.sum(a, axis=1)   np.sum(a, axis=0)
--------------             -----------------   --------------------
[[6, 3, 7, 4, 6],          [26, 28, 24, 18]    [22, 13, 27, 18, 16]
 [9, 2, 6, 7, 4],
 [3, 7, 7, 2, 5],
 [4, 1, 7, 5, 1]]
```

keepdims可以保持原来数组的**维数**。

```
np.sum(a,1,keepdims=True)  np.sum(a,0,keepdims=True)
---------------------------  ---------------------------
[[26],                          [[22, 13, 27, 18, 16]]
 [28],
 [24],
 [18]]
```

大小与排序

NumPy在排序等方面常用的函数如下：

函数名	功能	函数名	功能
min	最小值	max	最大值
ptp	极差	argmin	最小值的下标
mininum	二元最小值	maxinum	二元最大值
sort	数组排序	argsort	数组排序下标
percentile	分位数	median	中位数

min,max都有axis,out,keepdims等参数，我们来看其他函数。

```
> a = np.array([1, 3, 5, 7])
> b = np.array([2, 4, 6])
> np.maximum(a[None, :], b[:, None])#maxinum返回两组
array([[2, 3, 5, 7],                    矩阵广播计算后的
       [4, 4, 5, 7],                    结果
       [6, 6, 6, 7]])
```

大小与排序

sort()对数组进行排序会改变数组的内容，返回一个新的数组。**axis**的默认值都为-1，即按最终轴进行排序。axis=0对每列上的值进行排序。

```
np.sort(a)          np.sort(a, axis=0)
-----------------   ------------------
[[3, 4, 6, 6, 7],    [[3, 1, 6, 2, 1],
 [2, 4, 6, 7, 9],     [4, 2, 7, 4, 4],
 [2, 3, 5, 7, 7],     [6, 3, 7, 5, 5],
 [1, 1, 4, 5, 7]]     [9, 7, 7, 7, 6]]
```

percentile计算处于p%上的值。

```
> r = np.abs(np.random.randn(100000))
> np.percentile(r, [68.3, 95.4, 99.7])
array([ 1.00029686,  1.99473003,  2.9614485 ])
```

统计函数

NumPy中常用的统计函数有：**unique(), bicount(), histogram()**。我们来一个个介绍。首先看**unique():**

```
> np.random.seed(42)
> a = np.random.randint(0, 8, 10)
> np.unique(a)
a                                   np.unique(a)
------------------------------  ------------------
[6, 3, 4, 6, 2, 7, 4, 4, 6, 1]        [1, 2, 3, 4, 6, 7]
```

unique有两个参数,**return_index=True**同时返回原始数组中的下标,**return_inverse=True**表示原始数据在新数组的下标

```
> x, index = np.unique(a, return_index=True)]
x                          index                  a[index]
------------------  ------------------  ------------------
[1, 2, 3, 4, 6, 7]      [9, 4, 1, 2, 0, 5]    [1, 2, 3, 4, 6, 7]
```

统计函数

```
> x, rindex = np.unique(a, return_inverse=True)
 rindex                                  x[rindex]
------------------------------  ------------------------------
[4, 2, 3, 4, 1, 5, 3, 3, 4, 0]          [6, 3, 4, 6, 2, 7, 4, 4, 6, 1]
```

bincount()对**非负整数数组**中的各个元素出现的次数进行统计，返回数组中的第i个元素是整数i出现的次数。

```
> a = np.array([6, 3, 4, 6, 2, 7, 4, 4, 6, 1])
> np.bincount(a)
array([0, 1, 1, 1, 3, 0, 3, 1])

> x = np.array([0 ,  1,  2,  2,  1,  1,  0])
> w = np.array([0.1, 0.3, 0.2, 0.4, 0.5, 0.8, 1.2])
> np.bincount(x, w)
array([ 1.3,  1.6,  0.6])
```

统计函数

histogram()对以为数组进行直方图统计，其参数为：
histogram(a, bins=10, range=None, weights=None)
函数返回两个一维数组，**hist**是每个区间的统计结果，
bin_edges返回区间的边界值。

```
> a = np.random.rand(100)
> np.histogram(a, bins=5, range=(0, 1))
(array([28, 18, 17, 19, 18]),
 array([ 0. ,  0.2,  0.4,  0.6,  0.8,  1. ]))

> np.histogram(a, bins=[0, 0.4, 0.8, 1.0])
 (array([46, 36, 18]), array([ 0. ,  0.4,  0.8,  1. ]))
```

操作多维数组

多维数组可以进行**连接，分段**等多种操作。我们先来看**vstack(),hstack(),column_stack()**函数。

```
> a = np.arange(3)
> b = np.arange(10, 13)

> v = np.vstack((a, b)) # 按第1轴连接数组
> h = np.hstack((a, b)) # 按第0轴连接数组
> c = np.column_stack((a, b)) # 按列连接多个一维数组

 v                 h                          c
--------------  ------------------------  ----------
[[ 0,  1,  2],     [ 0,  1,  2, 10, 11, 12]     [[ 0, 10],
 [10, 11, 12]]                                   [ 1, 11],
                                                 [ 2, 12]]
```

操作多维数组

split()函数进行分段。

```
> a = np.array([6, 3, 7, 4, 6, 9, 2, 6, 7, 4, 3, 7])
> b = np.array([ 1,  3,  6,  9, 10]
> np.split(a, idx) # 按元素位置进行分段
[array([6]),
array([3, 7]),
array([4, 6, 9]),
array([2, 6, 7]),
array([4]),
array([3, 7])]

> np.split(a, 2) # 按数组个数进行分段
[array([6, 3, 7, 4, 6, 9]),
array([2, 6, 7, 4, 3, 7])]
```

多项式函数

多项式函数是整数的次幂与系数的乘积，如：
f(x)= an(x^n)+ an-1(x^(n-1))+...+ a1(x)+ a0
NumPy中多项式函数可以用**一维数组**表示。a[0]为最高次，a[-1]为常数项。

```
> a = np.array([1.0, 0, -2, 1])
> p = np.poly1d(a)
> print type(p)
<class 'numpy.lib.polynomial.poly1d'>

> p(np.linspace(0, 1, 5))
 array([ 1.      ,  0.515625,  0.125   , -0.078125,  0.      ])
```

多项式函数可以进行**四则运算**，其中**运算的列表自动化成多项式函数**。

多项式函数

```
> p + [-2, 1]
poly1d([ 1.,  0., -4.,  2.])

> p * p
poly1d([ 1.,  0., -4.,  2.,  4., -4.,  1.])

> p / [1, 1] # 分别为商和余
(poly1d([ 1., -1., -1.]), poly1d([ 2.]))
```

多项式也可以进行**积分和求导**。

```
> p.deriv()
poly1d([ 3.,  0., -2.])

> p.integ()
poly1d([ 0.25,  0.  , -1.  ,  1.  ,  0.  ])
```

多项式函数

Roots可以求多项式的根。

```
> r = np.roots(p)
> r
array([-1.61803399,  1.        ,  0.61803399])
```

polyfit()可以对数据进行多项式拟合。x, y为数据点，deg为多项式最高阶数。

```
> a = np.polyfit(x , y, deg)
```

poly()返回多项式系数构成的数组。

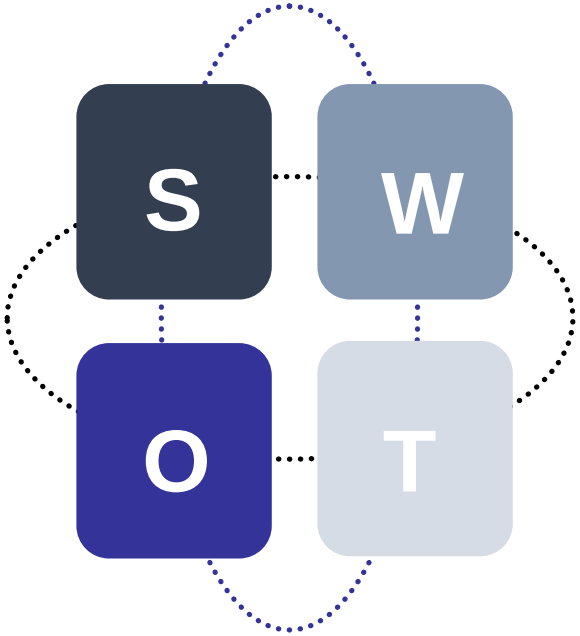
```
> a = np.poly(x )
```

参考文献

1. NumPy 官网 http://www.numpy.org/
2. NumPy 源代码：https://github.com/numpy/numpy

PyTorch简介

PyTorch具有先进设计理念的框架，其历史可追溯到2002年就诞生于纽约大学的Torch。Torch使用了一种不是很大众的语言Lua作为接口。Lua简洁高效，但由于其过于小众，以至于很多人听说要掌握Torch必须新学一门语言就望而却步，但Lua其实是一门比Python还简单的语言。



考虑到Python在计算科学领域的领先地位，以及其生态完整性和接口易用性，几乎任何框架都不可避免地要提供Python接口。终于，在2017年，Torch的幕后团队推出了PyTorch。PyTorch不是简单地封装Lua Torch提供Python接口，而是对Tensor之上的所有模块进行了重构，并新增了最先进的自动求导系统，成为当下最流行的动态图框架。

PyTorch的特点

01

PyTorch 既可以看作为加入了GPU 支持的numpy。TensorFlow 与Caffe 都是命令式的编程语言，而且是静态的，即首先必须构建一个神经网络，然后一次又一次使用同样的结构；如果想要改变网络的结构，就必须从头开始。但是PyTorch通过一种反向自动求导的技术，可以让用户零延迟地任意改变神经网络的行为。

02

PyTorch 的设计思路是线性、直观且易于使用的，当用户执行一行代码时，它会忠实地执行，所以当用户的代码出现Bug 的时候，可以通过这些信息轻松快捷地找到出错的代码，不会让用户在Debug 的时候因为错误的指向或者异步和不透明的引擎浪费太多的时间。

03

PyTorch 的代码相对于TensorFlow 而言，更加简洁直观，同时对于TensorFlow高度工业化的底层代码，PyTorch 的源代码就要友好得多，更容易看懂。深入API，理解PyTorch 底层肯定是一件令人高兴的事。

PyTorch概述

PyTorch 最大优势是建立的神经网络是动态的，可以非常容易的输出每一步的调试结果，相比于其他框架来说，调试起来十分方便。

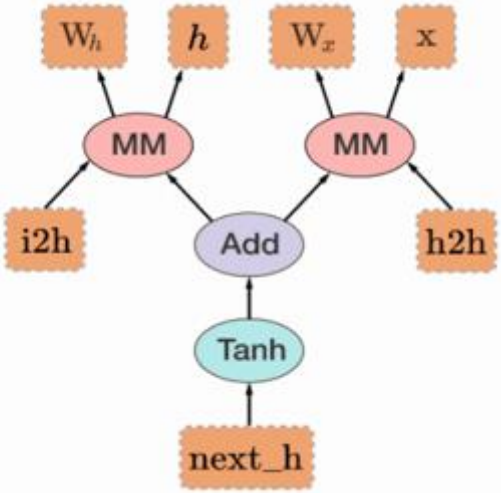
PyTorch的图是随着代码的运行逐步建立起来的，也就是说使用者并不需要在一开始就定义好全部的网络的结构，而是可以随着编码的进行来一点一点的调试，相比于TensorFlow和Caffe的静态图而言，这种设计显得更加贴近一般人的编码习惯。

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```

Back-propagation
uses the dynamically built graph

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()

next_h.backward(torch.ones(1, 20))
```



第三节 TensorFlow的特点和概述

TensorFlow简介

- TensorFlow 是一个采用数据流图（data flow graphs）用于数值计算的开源软件库。节点（Nodes）在图中表示数学操作，图中的线（edges）则表示在节点间相互联系的多维数据数组，即张量（tensor）。它灵活的架构让你可以在多种平台上展开计算，例如台式计算机中的一个或多个CPU（或GPU），服务器，移动设备等等。
- TensorFlow 最初由Google大脑小组的研究员和工程师们开发出来，用于机器学习和深度神经网络方面的研究，但这个系统的通用性使其也可广泛用于其他计算领域。

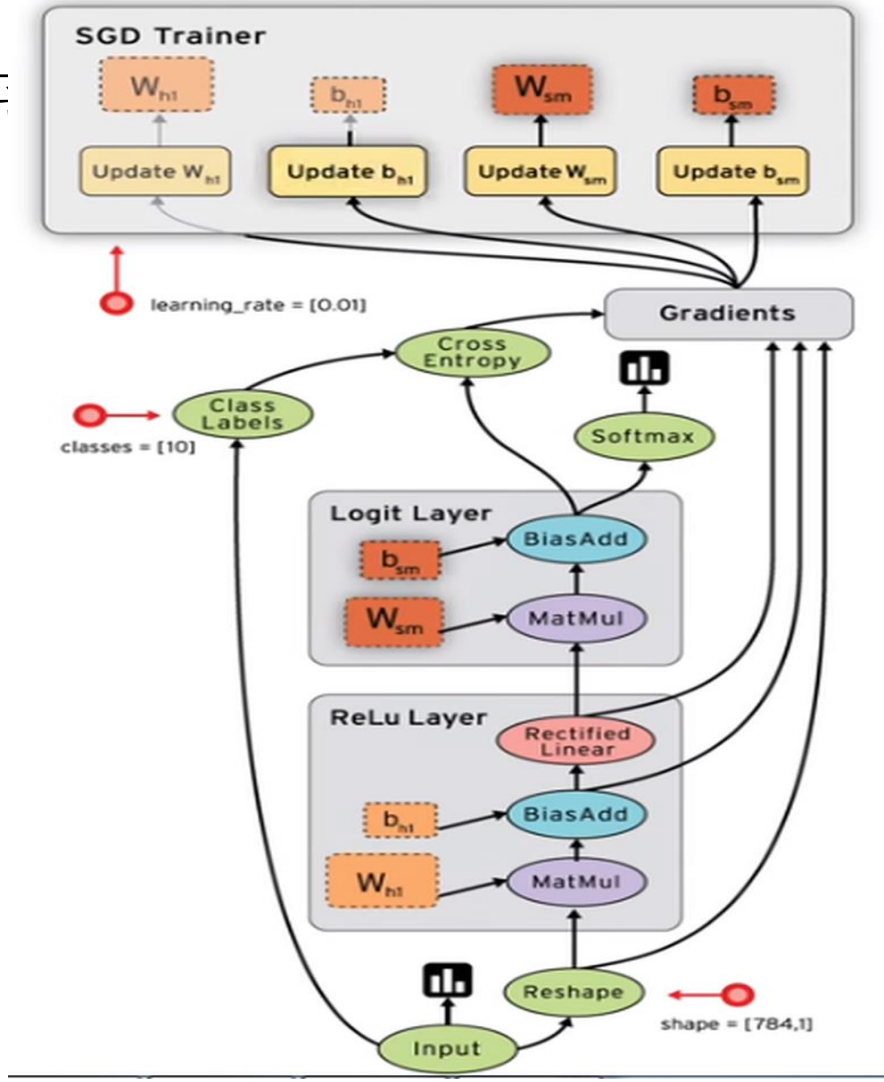
数据流图

节点

线

数据流

- 数据流图用“结点”（nodes）和“线”(edges)的有向图来描述数学计算。“节点”一般用来表示施加的数学操作，但也可以表示数据输入（feed in）的起点/输出（push out）的终点，或者是读取/写入持久变量（persistent variable）的终点。“线”表示“节点”之间的输入/输出关系。这些数据“线”可以输运“size可动态调整”的多维数据数组，即“张量”（tensor）。张量从图中流过的直观图像是这个工具取名为“TensorFlow”的原因。一旦输入端的所有张量准备好，节点将被分配到各种计算设备完成异步并行地执行运算。



TensorFlow的特点

特点

扩展便捷

TensorFlow 不是一个严格的“神经网络”库。只要用户可以将计算表示为一个数据流图就可以使用TensorFlow。用户负责构建图，描写驱动计算的内部循环。TensorFlow提供有用的工具来帮助用户组装“子图”，用户也可以在TensorFlow基础上写自己的“上层库”。TensorFlow的可扩展性相当强，可以自己写一些c++代码来丰富底层的操作。

数据透明

基于梯度的机器学习算法会受益于TensorFlow自动求微分的能力。只需要定义预测模型的结构，将这个结构和目标函数（objective function）结合在一起，并添加数据，TensorFlow将自动计算相关的微分导数。计算某个变量相对于其他变量的导数仅仅是通过扩展你的图来完成的，所以用户能一直清楚看到究竟在发生什么。

设备迁移

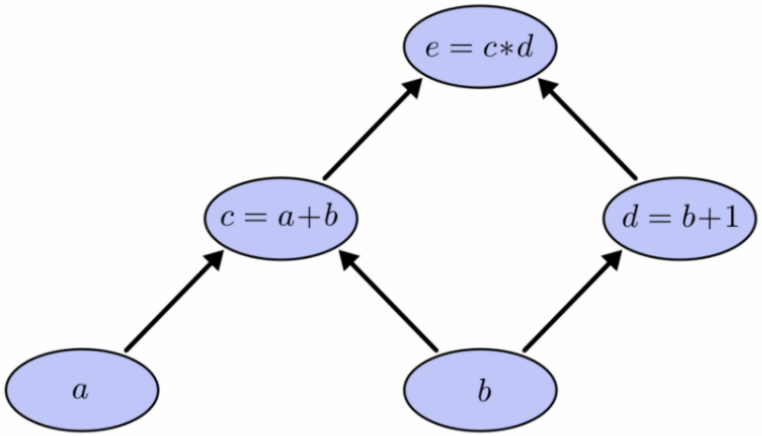
TensorFlow 在CPU和GPU上运行，比如说可以运行在台式机、服务器、手机移动设备等等。TensorFlow支持将训练模型自动在多个CPU上规模化运算，以及将模型迁移到移动端后台。

界面交互

TensorFlow 还有一个合理的c++使用界面，也有一个易用的python使用界面来构建和执行你的graphs。你可以直接写python/c++程序，也可以用交互式的ipython界面来用TensorFlow尝试些想法，它可以帮用户将笔记、代码、可视化等有条理地归置好。

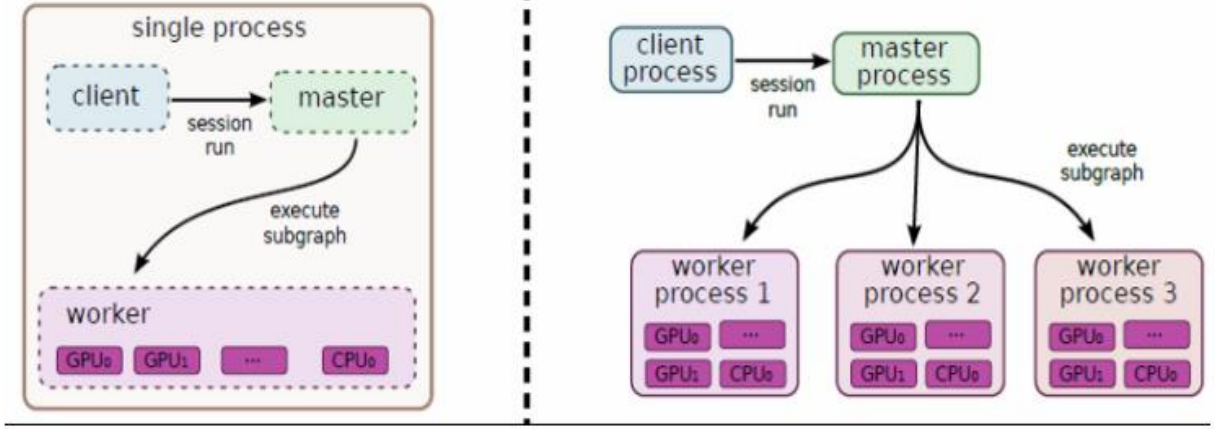
TensorFlow概述

- TensorFlow中Flow，也就是流，是其完成运算的基本方式。流是指一个计算图或简单的一个图，图不能形成环路，图中的每个节点代表一个操作，如加法、减法等。每个操作都会导致新的张量形成。
- 计算图具有以下属性：叶子顶点或起始顶点始终是张量。意即，操作永远不会发生在图的开头，由此可以推断，图中的每个操作都应该接受一个张量并产生一个新的张量。同样，张量不能作为非叶子节点出现，这意味着它们应始终作为输入提供给操作/节点。
- 计算图总是以层次顺序表达复杂的操作。通过将a + b表示为c，将b + 1表示为d，可以分层次组织上述表达式。 因此，可以将e写为：e = (c)x(d) 这里 c = a+b 且 d = b+1.以反序遍历图形而形成子表达式，这些子表达式组合起来形成最终表达式。
- 当我们正向遍历时，遇到的顶点总是成为下一个顶点的依赖关系，例如没有a和b就无法获得c，同样的，如果不解决c和d则无法获得e。同级节点的操作彼此独立，这是计算图的重要属性之一。
- 当我们按照图中所示的方式构造一个图时，很自然的是，在同一级中的节点，例如c和d，彼此独立，这意味着没有必要在计算d之前计算c。 因此它们可以并行执行。
- 计算图的并行也是最重要的属性之一。它清楚地表明，同级的节点是独立的，这意味着在c被计算之前不需空闲，可以在计算c的同时并行计算d。TensorFlow充分利用了这个属性。



TensorFlow概述

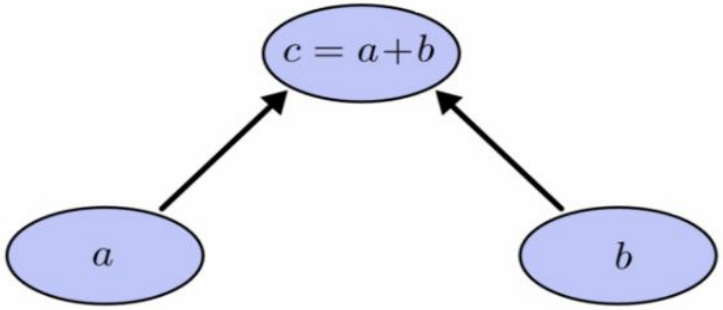
- TensorFlow允许用户使用并行计算设备更快地执行操作。计算的节点或操作自动调度进行并行计算。这一切都发生在内部，例如在上图中，可以在CPU上调度操作c，在GPU上调度操作d。下图展示了两种分布式执行的过程

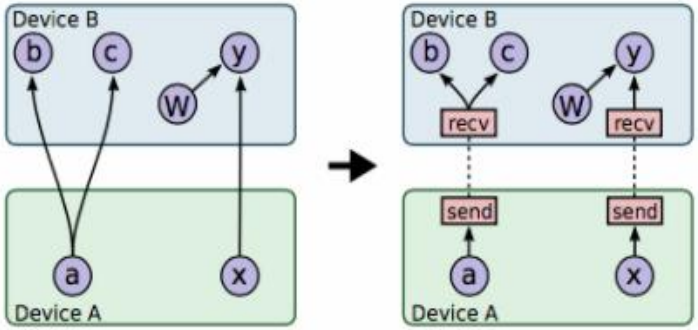


worker，并且该worker负责在各设备上调度任务。在第二种系统下，有多个worker，他们可以在同一台机器上或不同的机器上，每个worker都在自己的上下文中运行。在上图中，worker进程1运行在独立的机器上，并调度所有可用设备进行计算。

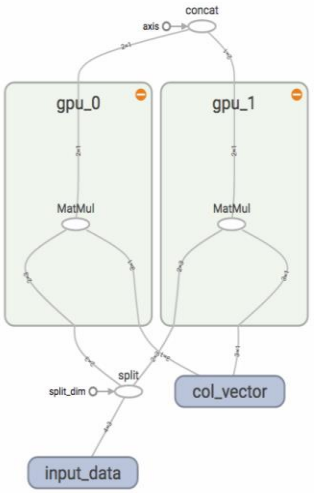
TensorFlow

- 子图是主图的一部分，从属性2可以说子图总是表示一个子表达式，因为c是e的子表达式。 子图也满足最后一个属性。同一级别的子图也相互独立，可以并行执行。因此可以在一台设备上调度整个子图。





TensorFlow将其所有操作分配到由worker管理的不同设备上。更常见的是，worker之间交换张量形式的数据，例如在e =（c）*（d）的图表中，一旦计算出c，就需要将其进一步传递给e，因此Tensor在节点间前向流动。该流动如右图所示。

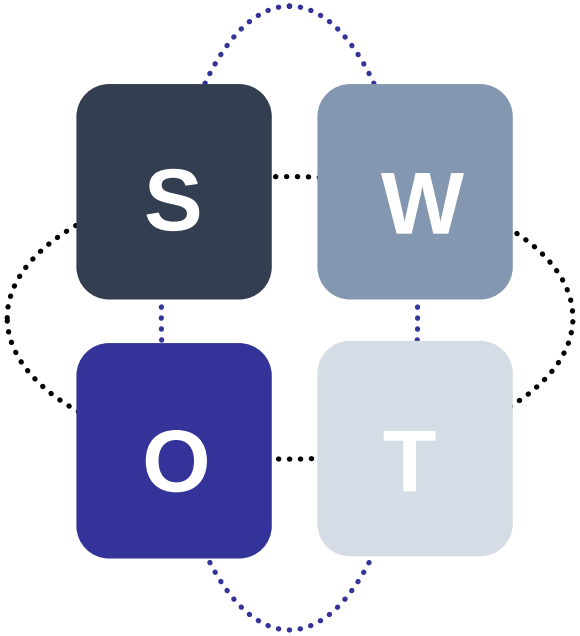


左图解释了子图的并行执行。这里有2个矩阵乘法运算，因为它们都处于同一级别，彼此独立，这符合最后一个属性。由于独立性的缘故，节点安排在不同的设备gpu_0和gpu_1上。

第四节 PaddlePaddle的特点和概述

PaddlePaddle简介

PaddlePaddle（飞桨）是由百度自主研发的开源深度学习框架。PaddlePaddle以百度多年的深度学习技术研究和业务应用为基础，集深度学习核心框架、基础模型库、端到端开发套件、工具组件和服务平台于一体。飞桨源于产业实践，始终致力于与产业深入融合。目前飞桨已广泛应用于工业、农业、服务业等，与合作伙伴一起帮助越来越多的行业完成 AI 赋能。飞桨2016年正式开源，2017年启动新一代深度学习框架研发，2018年 7月发布 Paddle v0.14，目前的最新版本是2022年底发布的Paddle v2.3.1。



PaddlePaddle的2.0版本相对 1.8 版本有重大升级，包括完善动态图功能，动态图模式下数据表示概念为Tensor；调整API 目录体系，API 的命名和别名进行了统一规范化；数据处理、组网方式、模型训练、多卡启动、模型保存和推理等开发流程都有了对应优化。

PaddlePaddle的特点

01

PaddlePaddle是灵活高效的产业级深度学习框架。它采用基于编程逻辑的组网范式，对于普通开发者而言更容易上手，符合他们的开发习惯。同时支持声明式和命令式编程，兼具开发的灵活性和高性能。网络结构自动设计，模型效果超越人类专家。

02

同时，PaddlePaddle支持超大规模深度学习模型的训练。它突破了超大规模深度学习模型训练技术，实现了世界首个支持千亿特征、万亿参数、数百节点的开源大规模训练平台，攻克了超大规模深度学习模型的在线学习难题，实现了万亿规模参数模型的实时更新。

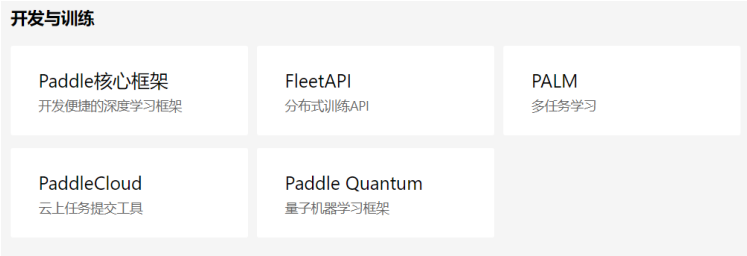
03

然后，PaddlePaddle还有多端多平台部署的高性能推理引擎。它不仅兼容其他开源框架训练的模型，还可以轻松地部署到不同架构的平台设备上。同时，飞桨的推理速度也是全面领先的。尤其经过了跟华为麒麟NPU的软硬一体优化，使得飞桨在NPU上的推理速度进一步突破。

PaddlePaddle概述

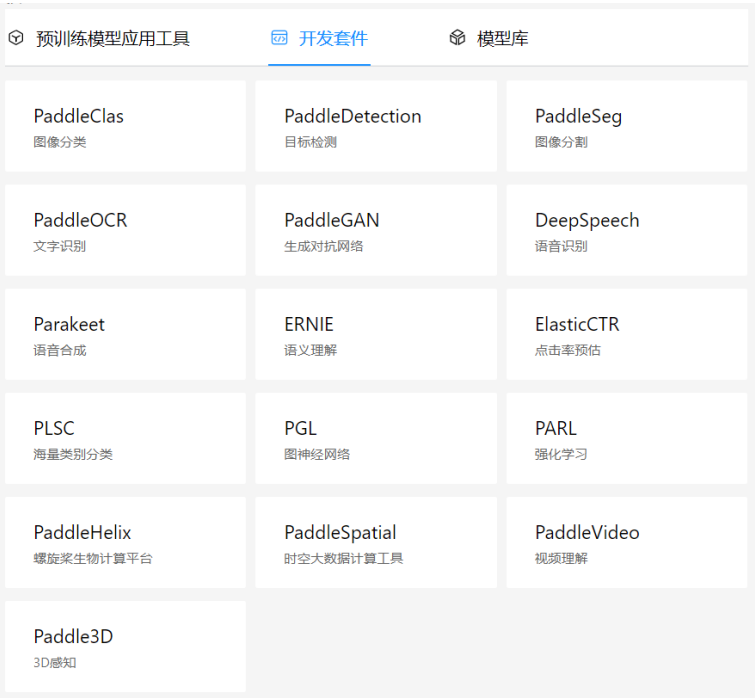
PaddlePaddle不仅仅是一个深度学习的框架，基于该核心框架，百度公司扩展了一系列的工具和模型，覆盖了深度学习开发和应用的整个流程，已然成为了一个功能齐全的产品。PaddlePaddle的官网上给出了产品的全景介绍，其中包括了主要使用流程和使用工具。

首先在开发和训练方面，如图 2 -10所示，PaddlePaddle以深度学习框架为核心，包括



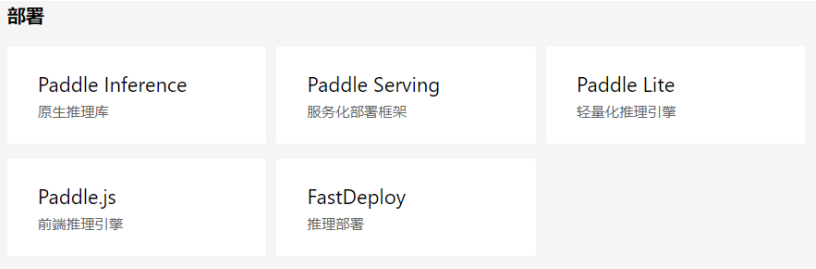
PaddlePaddle概述

在模型方面，PaddlePaddle提供了预训练模型工具PaddleHub和模型压缩工具PaddleSlim，同时为不同的领域提供了开发套件，如图 2- 11所示。尤其是PaddleOCR，在很多OCR的产品中都有其影子。同时还有文心大模型、PaddleCV、PaddleNLP等模型库，为开发者提供了极大的便利。



PaddlePaddle概述

而部署方面，PaddlePaddle也为不同的平台提供了相应工具。如图 2 -12所示，Paddle Inference提供原生推理库，Paddle Serving提供方便的服务端部署，Paddle Lite则面向轻量级部署，Paddle.js为Web前端提供部署方式，FastDeploy则是一个新的、综合的部署套件。



第五节 三者的比较

三者的比较

PyTorch

PyTorch是当前难得的简洁优雅且高效快速的框架。PyTorch的设计追求最少的封装，遵循tensor→variable(autograd)→nn.Module 三个由低到高的抽象层次，分别代表高维数组（张量）、自动求导（变量）和神经网络（层/模块），而且这三个抽象之间联系紧密，可以同时进行修改和操作。

PyTorch的灵活性不以速度为代价，在许多评测中，PyTorch的速度表现胜过TensorFlow和Keras等框架 。

简洁的设计带来的另外一个好处就是代码易于

三者的比较

PyTorch正在受到越来越多人的追捧 。

PyTorch是所有的框架中面向对象设计的最优雅的一个。PyTorch的面向对象的接口设计来源于Torch，而Torch的接口设计以灵活易用而著称。 PyTorch继承了Torch的衣钵，尤其是API的设计和模块的接口都与Torch高度一致。PyTorch的设计最符合人们的思维，它让用户尽可能地专注于实现自己的想法，即所思即所得，不需要考虑太多关于框架本身的束缚。

PyTorch提供了完整的文档，循序渐进的指南，作者亲自维护的论坛 供用户交流和求教问题。Facebook 人工智能研究院对PyTorch提供了强力支持，作为当今排名前三的深度学习研究机构，FAIR的支持足以确保PyTorch获得持续的开发更新。

同样的，由于推出时间较短，在Github上并没有如Caffe或TensorFlow那样多的代码实现，使用TensorFlow能找到很多别人的代码，而对于PyTorch的使用者，可能需要自己完成很多的代码实现。

三者的比较

TensorFlow

TensorFlow已经成为当今最炙手可热的深度学习框架，但是由于自身的缺陷，TensorFlow离最初的设计目标还很遥远。

TensorFlow编程接口支持Python和C++。随着1.0版本的公布，Java、Go、R和Haskell API的alpha版本也被支持。此外，TensorFlow还可在Google Cloud和AWS中运行。TensorFlow还支持 Windows 7、Windows 10和Windows Server 2016。由于TensorFlow使用C++ Eigen库，所以库可在ARM架构上编译和优化。这也就意味着用户可以在各种服务器和移动设备上部署自己的训练模型，无须执行单独的模型解码器或者加载Python解释器。

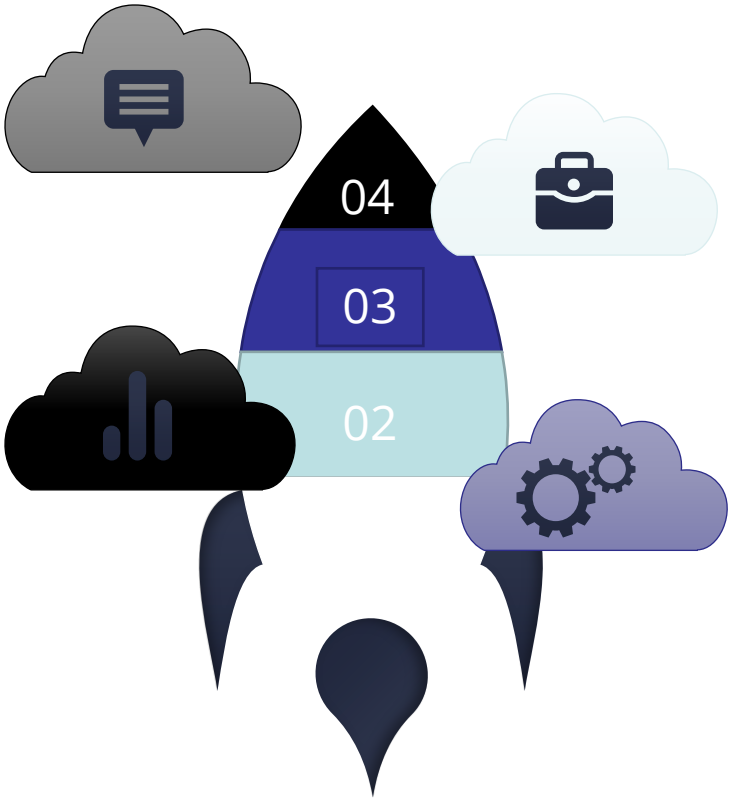
TensorFlow和Theano拥有相近的设计理念，都是基于计算图实现自动微分系统。TensorFlow 使用数据流图进行数值计算，图中的节点代表数学运算， 而图中的边则代表在这些节点之间传递的多维数组。

三者的比较

TensorFlow的批评

过于复杂的系统设计。TensorFlow 代码量庞大，使维护难以完成。对于入门者，TensorFlow的底层运行机制难以掌握，很多时候导致了放弃。

接口设计过于晦涩难懂，拥有图、会话、命名空间、PlaceHolder等诸多抽象概念，对普通用户来说难以理解。同一个功能，TensorFlow提供了多种实现，这些实现良莠不齐，使用中还有细微的区别，很容易将用户带入坑中。



频繁变动的接口。TensorFlow的接口一直处于快速迭代之中，且向后兼容性较差，这导致许多陈旧的开源代码已经无法在新版运行，同时也间接导致了许多第三方框架出现BUG。至今TensorFlow仍没有一个统一易用的接口。

TensorFlow作为一个复杂的系统，文档和教程众多，但缺乏明显的条理和层次，虽然查找很方便，但用户却很难找到一个真正循序渐进的入门教程。

三者的比较

PaddlePaddle

PaddlePaddle给我们留下最深的印象就是巨大的本土化优势。现在的深度学习框架的发展的速度逐渐慢了下来，这也意味着深度学习框架在现阶段已经趋近成熟。而我们在使用框架时会发现，框架上的差异逐渐小了，API也越来越相似，甚至框架的迁移也变得容易起来。但是以PaddlePaddle为代表的国产深度学习框架则在发展中不断寻找着属于自己的特色。

PaddlePaddle

PaddlePaddle有着极其详细的中文教程、开发工具，同时也会不定期组织开发者培训，组建开发者讨论社区，为开发者提供极大的便利。其它的国产深度学习框架也各显特色，如华为的MindSpore的倡导AI算法即代码，降低AI开发门槛，基于数学原生表达的AI编程新范式让算法专家聚焦AI创新和探索；清华大学发布的计图（Jittor）是一个完全基于动态编译（Just-in-time）、内部使用创新的元算子和统一计算图的深度学习框架。我们希望看到一个百家争鸣的环境，这样才会因循守旧，才会共同进步。。

小结

本章介绍了三种常用的机器学习框架，其中TensorFlow和PyTorch是目前最流行的两种开源框架。在以往版本的实现中，TensorFlow最开始提供静态图构建的功能，具有较高的运算性能，但是模型的调试分析成本较高；之后在TensorFlow2.x版本中提供动态图，提高了开发效率。PyTorch主要提供动态图计算的功能，API涉及接近Python原生语法，因此易用性较好，但是在图形优化方面不如TensorFlow。这样的特点使TensorFlow被大量用于AI企业的模型部署，而学术界则大量使用PyTorch进行研究。不过目前我们也看到两种框架正在吸收对方的优势，如TensorFlow的eager模式就是对动态图的一种尝试。另外，国内的深度学习框架发展也很迅猛，以百度的PaddlePaddle为代表的国产深度学习框架在功能上也不输PyTorch和TensorFlow，而且有更多的本土特色，很适合国内的开发者研究学习和使用。同样，还有很多其它的国产深度学习框架也值得学习，例如华为的MindSpore、旷视的MegEngine、清华大学的Jittor，有兴趣的读者可以深入了解。

文档论文写作工具Latex

https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes

- LE_IJCV
- LE_MM23
  - ablat.png
  - ablation.png
  - data.png
  - distribution.png

File outline

Introduction
Related Work
Unlearnable Examples
Diffusion Models
Methodology
Problem Statement
Data Protector.
Unauthorized Data Exploiter.
Learnable Examples
Joint-conditional Diffusion Purification
DDPM for Data Purified.
Joint-conditional Diffusion Model.
Fine-tuning for Diffusion Model.
Experiments
Settings
Datasets and Evaluation.
Training Details.
Evaluation on Supervised UEs
Comparsion with AT Methods and Dat...
Comparsion with Image Compression.
Comparsion with Noise Purification.
Evaluation on Unsupervised UEs
Investigating the Distribution Similarity
Ablation Studies
Conclusions

```
% \received[revised]{12 March 2009}
% \received[accepted]{5 June 2009}

%%
%% This command processes the author and affiliation and title
%% information and builds the first part of the formatted document.
\maketitle

\section{Introduction}
The abundance of ``freely'' accessible data on the Web has been pivotal to the success of
modern deep learning, such as ImageNet~\cite{russakovsky2015imagenet} and Ms-celeb-
1m~\cite{Guo2016MSCeleb1MAD}. However, these datasets might include personal data
collected without mutual consent~\cite{birhane2021large}, which has raised public concerns
that private data can be utilized to create commercial models without the owner's
authorization~\cite{hill2019photos}. To address such concerns, growing
efforts~\cite{RUE,HuangUE,yu2022availability} have been made to add protection to data to
prevent unauthorized usage by making the data unexploitable. These methods add
imperceptible ``shortcut'' noise to the images so that the deep learning models learn no
useful semantics but correspondences between noise and labels~\cite{geirhos2020shortcut}.
Consequently, the models trained on unexploitable data fail to classify clean data,
thereby safeguarding users' privacy. Such poisoning methods are named as unlearnable
example (UE) protection~\cite{HuangUE} or availability attack~\cite{yu2022availability}.

% Despite the increasing number of studies on unexploitable
data~\cite{sankar2023cuda,zhang2022unlearnable,ren2022transferable,fowl2021adversarial,wang
2021fooling,he2022indiscriminate}, its inherent vulnerabilities are often overloodked.
% A widely adopted belief is that unlearnable examples can safeguard users' privacy.
However, we find that this sense of security may be illusory, as this strategy ignores an
inherent asymmetry between parties: data publishers can only add the``unlearnable"
perturbations to the training data, but they cannot prevent unauthorized third parties
from collecting clean raw data in the wild. For instance, let us assume that an
unlearnable version of dataset for visual recognition has been made public. An
unauthorized corporate entity attempts to train neural networks on the unauthorized data.
In the evaluation stage, the company needs to collect testing data to evaluate whether the
unauthorized trained model generalizes well on the original data distribution, while the
data publisher cannot prevent unauthorized third parties from collecting raw data in the
wild. A vital vulnerability implicit in this asymmetry is the unauthorized entity can
easily bypass the protected training data to learn the original data representation from
the collected raw testing data, even if the testing data might be small in size scale and
lacks label annotation~\cite{wang2018transferring,closer_fewshot}.

While the growing research focuses on how to make data
unexploitable~\cite{sankar2023cuda,zhang2022unlearnable,ren2022transferable,fowl2021adversa
rial,wang2021fooling,he2022indiscriminate}, we aim to challenge this paradigm by exposing
a key vulnerability in this protection: the protection is merely effective if the
unexploitable data is all that is accessible. Unfortunately, this is often not the case.
Data protectors can only add the ``unlearnable" perturbations to their own data, but they
cannot prevent unauthorized users from accessing similar, unprotected data from other
```

Unlearnable Examples Give a False Sense of Security: Piercing through Unexploitable Data with Learnable Examples

Wan Jiang*
Hefei University of Technology
Hefei, China

Yunfeng Diao*†
Hefei University of Technology
Hefei, China

He Wang†
University College London
London, United Kingdom

Jianxin Sun
Chinese Academy of Science
Beijing, China

Meng Wang
Hefei University of Technology
Hefei, China

Richang Hong
Hefei University of Technology
Hefei, China

ABSTRACT

Safeguarding data from unauthorized exploitation is vital for privacy and security, especially in recent rampant research in security breach such as adversarial/membership attacks. To this end, *unlearnable examples* (UEs) have been recently proposed as a compelling protection, by adding imperceptible perturbation to data so that models trained on them cannot classify them accurately on original clean distribution. Unfortunately, we find UEs provide a false sense of security, because they cannot stop unauthorized users from utilizing other unprotected data to remove the protection, by turning unlearnable data into learnable again. Motivated by this observation, we formally define a new threat by introducing *learnable unauthorized examples* (LEs) which are UEs with their protection removed. The core of this approach is a novel purification process that projects UEs onto the manifold of LEs. This is realized by a new joint-conditional diffusion model which denoises UEs conditioned on the pixel and perceptual similarity between UEs and LEs. Extensive experiments demonstrate that LE delivers state-of-the-art countering performance against both supervised UEs and unsupervised UEs in various scenarios, which is the first generalizable countermeasure to UEs across supervised learning and unsupervised learning. Our code is available at https://github.com/jiangw-0/LE_JCDP.

CCS CONCEPTS

• **Security and privacy** → **Human and societal aspects of security and privacy**; • **Computing methodologies** → **Machine learning**.

KEYWORDS

Unlearnable Examples, Data Protection, Deep Neural Network

*Both authors contributed equally to this research.
†Corresponding authors. E-mails: diaoyunfeng@hfut.edu.cn, he_wang@ucl.ac.uk

ACM Reference Format:
Wan Jiang, Yunfeng Diao, He Wang, Jianxin Sun, Meng Wang, and Richang Hong. 2023. Unlearnable Examples Give a False Sense of Security: Piercing through Unexploitable Data with Learnable Examples. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23), October 29–November 3, 2023, Ottawa, ON, Canada.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3581783.3611833

1 INTRODUCTION

The abundance of "freely" accessible data on the Web has been pivotal to the success of modern deep learning, such as ImageNet [37] and Ms-celeb-1m [15]. However, these datasets might include personal data collected without mutual consent [2], which has raised public concerns that private data can be utilized to create commercial models without the owner's authorization [19]. To address such concerns, growing efforts [11, 21, 47] have been made to add protection to data to prevent unauthorized usage by making the data unexploitable. These methods add imperceptible "shortcut" noise to the images so that the deep learning models learn no useful semantics but correspondences between noise and labels [12]. Consequently, the models trained on unexploitable data fail to classify clean data, thereby safeguarding users' privacy. Such poisoning methods are named as unlearnable example (UE) protection [21] or availability attack [47].

While the growing research focuses on how to make data unexploitable [10, 16, 35, 38, 43, 51], we aim to challenge this paradigm by exposing a key vulnerability in this protection: the protection is merely effective if the unexploitable data is all that is accessible. Unfortunately, this is often not the case. Data protectors can only add the "unlearnable" perturbations to their own data, but they cannot prevent unauthorized users from accessing similar, unprotected data from other sources. As a result, one can study the underlying distribution of the protected examples, via studying similar newly collected (unprotected) data. Taking face recognition as an example, although unlearnable examples cannot be directly used to

```
1}|\boldsymbol{x}_{t}) $  in \cref{eq:reverse}, taking into account the premise assumption
mentioned earlier that clean samples cannot be obtained and that the difference between
unlearnable and clean samples is imperceptible. Then, by conditioning this process
$p_{\varphi}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_{t}) $ on $\Tilde{\boldsymbol{x}}_t $ and
$\Psi({\Tilde{\boldsymbol{x}}_t})$, we can sample $\boldsymbol{x}_{t-1}$ according to
$p_{\varphi}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_{t}, \Tilde{\boldsymbol{x}}_t,
\Psi({\Tilde{\boldsymbol{x}}_t})) $ and eventually obtain denoised learnable samples. In
other words, we use the reverse denoising process to restore the original clean signal
while considering the small and imperceptible differences between unlearnable and clean
data. Among them, $\Tilde{\boldsymbol{x}}_t$ is obtained from the unlearnable sample
$\Tilde{\boldsymbol{x}}_0$ through $t$-step diffusion, and
$\Psi({\Tilde{\boldsymbol{x}}_t})$ is the human perception of $\Tilde{\boldsymbol{x}}_t$.
Obviously, it is easy to get the following equation:
%}
\begin{equation}
\begin{split}
\label{eq:joint}
    &p_{\varphi}(\Tilde{\boldsymbol{x}}_{t-1}^* |\Tilde{\boldsymbol{x}}_{t}^* ,
    \Tilde{\boldsymbol{x}}_t, \Phi({\Tilde{\boldsymbol{x}}_t})) \\
    & =  \frac{p(\Phi({\Tilde{\boldsymbol{x}}_t})|\Tilde{\boldsymbol{x}}_{t-1}^* ,
    \Tilde{\boldsymbol{x}}_t^* , \Tilde{\boldsymbol{x}}_t ) p_{\varphi}
    (\Tilde{\boldsymbol{x}}_{t-1}^* |\boldsymbol{x}_{t}^* , \Tilde{\boldsymbol{x}}_t )}{p(
    \Phi({\Tilde{\boldsymbol{x}}_t})| \Tilde{\boldsymbol{x}}_t^* ,
    \Tilde{\boldsymbol{x}}_t )} \\
    & = p(  \Tilde{\boldsymbol{x}}_{t}^*    |\Tilde{\boldsymbol{x}}_{t-1}^* ,
    \Tilde{\boldsymbol{x}}_t, \Phi({\Tilde{\boldsymbol{x}}_t})
    )\frac{p(\Phi({\Tilde{\boldsymbol{x}}_t})|\Tilde{\boldsymbol{x}}_{t-1}^* ,
    \Tilde{\boldsymbol{x}}_t )}{p( \Tilde{\boldsymbol{x}}_{t}^* |\Tilde{\boldsymbol{x}}_{t-
    1}^*, \Tilde{\boldsymbol{x}}_t )} \frac{p_{\varphi} (\Tilde{\boldsymbol{x}}_{t-1}^*
    |\Tilde{\boldsymbol{x}}_{t}^*, \Tilde{\boldsymbol{x}}_t )}{p(
    \Phi({\Tilde{\boldsymbol{x}}_t})|\Tilde{\boldsymbol{x}}_{t}^*,
    \Tilde{\boldsymbol{x}}_t )}
    % & = \frac{p(\Phi({\Tilde{\boldsymbol{x}}_t})|\boldsymbol{x}_{t-1},
    \Tilde{\boldsymbol{x}}_t ) p(\boldsymbol{x}_{t-1} | \boldsymbol{x}_{t},
    \Tilde{\boldsymbol{x}}_t )} {p(\Phi({\Tilde{\boldsymbol{x}}_t})|\boldsymbol{x}_{t},
    \Tilde{\boldsymbol{x}}_t )}
\end{split}
\end{equation}
Since the diffuse process of DDPM is a Markov process, we have: % \YF{Eq11. needs to be
explained further}
\begin{equation}
```


the joint conditional distribution $p_\varphi(\tilde{x}^*_{0:T_p}|\tilde{x}, \Phi(\tilde{x}))$:

$$p_\varphi(\tilde{x}^*_{0:T_p}|\tilde{x}, \Phi(\tilde{x})) = p(\tilde{x}^*_{T_p}) \prod_{t=1}^{T} p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t)) \tag{10}$$

where $\tilde{x}_t$ is the noise version of $\tilde{x}_0$ via t-step diffusion. Using Bayes' rule, we can derive the joint conditional reverse process:

$$\begin{aligned} &p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t)) \\ &= \frac{p(\Phi(\tilde{x}_t)|\tilde{x}^*_{t-1}, \tilde{x}^*_t, \tilde{x}_t) p_\varphi(\tilde{x}^*_{t-1}|x^*_t, \tilde{x}_t)}{p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t)} \\ &= p(\tilde{x}^*_t|\tilde{x}^*_{t-1}, \tilde{x}_t, \Phi(\tilde{x}_t)) \frac{p(\Phi(\tilde{x}_t)|\tilde{x}^*_{t-1}, \tilde{x}_t)}{p(\tilde{x}^*_t|\tilde{x}^*_{t-1}, \tilde{x}_t)} \frac{p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t)}{p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t)} \end{aligned} \tag{11}$$

Since the diffuse process of DDPM is a Markov process, we have:

$$p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t)) = \frac{p(\Phi(\tilde{x}_t)|\tilde{x}^*_{t-1}, \tilde{x}_t) p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t)}{p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t)} \tag{12}$$

then we take the logarithm of both sides

$$\begin{aligned} \log p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t,\ \tilde{x}_t, \Phi(\tilde{x}_t)) &= \log p(\Phi(\tilde{x}_t)|\tilde{x}^*_{t-1}, \tilde{x}_t) \\ &+ \log p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t) - \log p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t) \end{aligned} \tag{13}$$

We can approximate $\log p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t)$ using a Taylor expansion around $\tilde{x}^*_t = \tilde{x}^*_{t-1}$. This gives

$$\begin{aligned} &\log p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t)) \\ &\approx (\tilde{x}^*_{t-1} - \tilde{x}^*_t) \nabla_{\tilde{x}^*_t} \log p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t) + \log p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t) \end{aligned} \tag{14}$$

According to work [39] and [6], it is proved that:

$$\begin{aligned} &p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t) \\ &\approx \mathcal{N}(\tilde{x}^*_{t-1}; \mu_\varphi(\tilde{x}^*_t, t) + \sigma_t^2 \nabla_{\tilde{x}^*_t} \log p(\tilde{x}_t|\tilde{x}^*_t), \sigma_t^2 I) \end{aligned} \tag{15}$$

then we can show that

$$\begin{aligned} &p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t)) \\ &\approx \mathcal{N}(\tilde{x}^*_{t-1}; \mu_\varphi(\tilde{x}^*_t, t) + \sigma_t^2(\boldsymbol{d}_1 + \boldsymbol{d}_2), \sigma_t^2 I) \\ &\boldsymbol{d}_1 = \nabla_{\tilde{x}^*_t} \log p(\tilde{x}_t|\tilde{x}^*_t), \boldsymbol{d}_2 = \nabla_{\tilde{x}^*_t} \log p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t) \end{aligned} \tag{16}$$

In the above equation, $p(\tilde{x}_t|\tilde{x}^*_t)$ and $p(\Phi(\tilde{x}_t)|\tilde{x}^*_t, \tilde{x}_t)$ represent how likely $\tilde{x}^*_t$ is close to $\tilde{x}_t$ under the data space and latent feature space in the reverse process. Following [41], we can utilize MSE as the distance metric $\mathcal{D}_{\boldsymbol{m}}$ to approximate $p(\tilde{x}_t|\tilde{x}^*_t)$:

$$p(\tilde{x}_t|\tilde{x}^*_t) = \frac{1}{Z} exp(\lambda_1 \mathcal{D}_{\boldsymbol{m}}(\tilde{x}^*_t, \tilde{x}_t)) \tag{17}$$

Based on the inference above, the conditional transition operator $p_\varphi(\tilde{x}^*_{t-1}|\tilde{x}^*_t, \tilde{x}_t, \Phi(\tilde{x}_t))$ can be approximated by a Gaussian distribution, whose mean is shifted by $\sigma_t^2(\boldsymbol{d}_1 + \boldsymbol{d}_2)$. Algorithm 1 outlines the inference details of JCDP.

Algorithm 1: Joint-conditional Diffusion Purification

Input: Unlearnable example $\tilde{x}_0$, diffusion step T_p per each purification run, number of purification iterations N, given a DDPM $(\mu_\varphi(x_t, t), \sigma_t^2 I)$, gradient scale λ_1 and λ_2.

Init: $\boldsymbol{d}_1 = 0$ and $\boldsymbol{d}_2 = 0$;
for $i \leftarrow 1$ *to* N **do**
  The diffusion process:
  $q(\tilde{x}_{1:T_p}|\tilde{x}_0) = \prod_{t=1}^{T_p} q(\tilde{x}_t|\tilde{x}_{t-1})$;
  The reverse process:
  for $t \leftarrow T_p$ *to* 1 **do**
    $\mu, \sigma_t^2 \leftarrow \mu_\varphi(\tilde{x}^*_t, t), \sigma_t^2$;
    $\boldsymbol{d}_1 \leftarrow -\lambda_1 \nabla_{\tilde{x}^*_t} \mathcal{D}_{\boldsymbol{m}}(\tilde{x}^*_t, \tilde{x}_t)$;
    $\boldsymbol{d}_2 \leftarrow -\lambda_2 \nabla_{\tilde{x}^*_t} \mathcal{D}_{\boldsymbol{p}}(\tilde{x}^*_t, \tilde{x}_t)$;
    $\tilde{x}^*_{t-1} \leftarrow$ sample from $\mathcal{N}(\mu + \sigma_t^2(\boldsymbol{d}_1 + \boldsymbol{d}_2), \sigma_t^2 I)$
  end
end
return $\tilde{x}^*_0$

3.3.3 Fine-tuning for Diffusion Model. We can directly train a diffusion model on newly collected (unprotected) data. However, since the size of the collected data is typically much smaller than the original training data, the generation quality may be low. Fortunately, it is almost impossible that data protectors add unlearnable perturbations to all data in the real world. Therefore, we can transfer knowledge from unprotected source domains to unauthorized target domains with limited collected data by means of fine-tuning. Specially, we first initialize DDPM with the weights of a source network pre-trained on one unprotected domain, and then fine-tune it on the newly collected data. This simple fine-tuning operation can shorten the convergence time and improve the purification performance, especially when collected raw data is limited.

4 EXPERIMENTS

4.1 Settings

```latex
\begin{table}[!tb]
\caption{Comparsion with ISS on CIFAR-10 and CIFAR-100.}
\label{tab:iss}
% \vskip 0.15in
\begin{center}
\begin{small}
\begin{sc}
\resizebox{1\linewidth}{!}{
\begin{tabular}{l|l|ccccc}
\toprule
Data&UE &Vanila &BGR&Gray&JPEG&LE (Ours)\\
\midrule
\multirow{4}{*}{CIFAR-10} &Clean  &94.7 &88.7 &92.4 &85.4 &n/a\\
& EMN &21.2 &36.5 &93.0 &81.5 &\textbf{93.1}\\
&REMN &20.5 &40.8 &\textbf{92.8} &82.3 &92.2\\
&LSP &15.0 &66.2 &82.5 &83.1 &\textbf{92.4}\\
\midrule
\multirow{4}{*}{CIFAR-100} &Clean &74.5 &- &71.8 &57.8 &n/a\\
& EMN &14.8 &- &67.5 &56.0 &\textbf{70.9}\\
&REMN &10.9 &- &57.3 &55.8 &\textbf{65.3}\\
&LSP &4.1 &- &44.6 &53.5 &\textbf{68.7}\\
\bottomrule
\end{tabular}
}
\end{sc}
\end{small}
\end{center}
% \vskip -0.1in
\end{table}
```


Table 3: Comparsion with ISS on CIFAR-10 and CIFAR-100.

Data	UE	Vanila	BGR	Gray	JPEG	LE (Ours)
CIFAR-10	Clean	94.7	88.7	92.4	85.4	n/a
	EMN	21.2	36.5	93.0	81.5	**93.1**
	REMN	20.5	40.8	**92.8**	82.3	92.2
	LSP	15.0	66.2	82.5	83.1	**92.4**
CIFAR-100	Clean	74.5	-	71.8	57.8	n/a
	EMN	14.8	-	67.5	56.0	**70.9**
	REMN	10.9	-	57.3	55.8	**65.3**
	LSP	4.1	-	44.6	53.5	**68.7**

Comparsion with Image Compression. Image Shortcut Squeezing (ISS) contains three common image compression operations, including grayscale transformation (GRAY), JPEG and bit depth reduction (BDR). We compare LE with every operation in ISS and the results are shown in Tab. 3. Different from LS showing consistent countering performance across all UE protections, the optimal compression operation in ISS varies across different types of UEs, e.g. JPEG is not effective against error-minimizing noise (EMN, REMN) while GRAY is not effective against patch-based linear separable perturbation (LSP). In order to get the optimal performance, ISS has to ensemble multiple models which are applied with different compressions. As a result, LE is more efficient than ISS and has the best overall performance.

Comparsion with Noise Purification. Similar to LE, AVATAR [7] also uses diffusion model for purifying unlearnable perturbation. However, the main experiments reported in [7] utilize the original clean training data for training diffusion models. In contrast, our

Table 4: Performance of LE against unsupervised UE (CP&UC). Table report linear probing accuracy. The best performance is shown in bold.

UE	Data	Backbone	Clean	Vanila	LE (Ours)
CP [16]	CIFAR-10	SimCLR	90.4	44.9	**86.6**
		MoCo v2	89.3	55.1	**86.0**
		BYOL	92.2	59.6	**85.7**
	CIFAR-100	SimCLR	63.6	34.7	**57.4**
		MoCo v2	65.2	41.9	**57.1**
		BYOL	65.3	39.2	**57.2**
UC [51]	Pets	SimCLR	48.3	17.9	**46.0**
UC-CLIP [51]				26.1	**47.5**

for unsupervised setting, but are robustness to unsupervised exploitation against SimCLR. We evaluate the effectiveness of LE on SimCLR for UC. As shown in Tab. 4, LE is consistently effective across various UCL algorithms. As there is no unsupervised countering method for direct comparison, we employ the contrastive learning version of AT (AdvCL) [8] and common data augmentations like Cutout [5] and Random Noise. We conduct comparisons using backbone SimCLR on CIFAR-10. The linear probing accuracy of AdvCL, Cutout and Random Noise is 79.3%, 47.7%, and 54.1% respectively, while LE achieves 86.6%, showing the superiority of LE by big margins in the context of unsupervised learning. To the best of our knowledge, LE is the first generalizable countermeasure that is effective against UEs in both supervised and unsupervised learning.

4.4 Investigating the Distribution Similarity